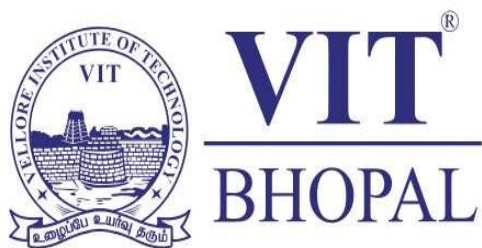


VITYARTHI – PROJECT



Project title: ROCK · PAPER · SCISSORS – CLI GAME

Course title: Introduction to Problem Solving & Programming

Course code: CSE 1021

Course type: Flipped course

Course credits: 4

Professor: Sivabalan KR

Slot: B21+B22+B23+E14+E21+E22

Submitted by: M S Bharani

Registration number: 26BOE10022

Submission date: 30/09/2026

INTRODUCTION:

VITYARTHI – PROJECT

The **Rock · Paper · Scissors CLI Game** is a simple interactive command-line game developed using Python. The project allows a user to play the traditional Rock–Paper–Scissors game against a computer-controlled opponent referred to as the AI.

The game is designed to provide a simple and engaging terminal-based experience while demonstrating important Python programming concepts such as functions, loops, conditional statements, dictionaries, input handling, random number generation, exception handling, and score management.

At the beginning of a match, the player is asked to enter the target number of points required to win. The target must be a positive integer. Once the target is selected, the game starts and continues until either the player or the AI reaches the target score.

During every round, the player can select Rock, Paper, or Scissors using the options 1, 2, or 3. The computer independently selects one of the three choices using Python's `random.choice()` function. The selected choices are displayed along with corresponding icons.

The game then compares the player's choice with the AI's choice. If both choices are the same, the round is declared a tie. Otherwise, the standard Rock–Paper–Scissors rules determine the winner. The winning player receives one point.

The Python source code uses a function-based design with separate functions for obtaining the target score and running the main game. The random module is used to generate the

VITYARTHI – PROJECT

AI's choice, while the sys module is used to terminate the program when the user chooses to quit.

PROBLEM STATEMENT:

Traditional Rock–Paper–Scissors is normally played between two people. In a computer-based version, the system must provide an opponent that can make a choice independently, compare the choices, maintain the score, and determine the final winner.

A simple implementation also needs to handle incorrect user input. For example, the player may enter a character instead of a number, enter a number outside the available options, or provide an invalid target score.

The objective of this project is therefore to develop a lightweight command-line game that allows a human player to compete against a computer-controlled opponent while maintaining scores until a predefined target is reached.

The system should provide:

- A clear command-line interface.
- A configurable target score.
- Three player choices: Rock, Paper, and Scissors.
- Random AI selection.
- Round-by-round result calculation.
- Continuous score tracking.
- Invalid input handling.
- A quit option.
- A final match result.

VITYARTHI – PROJECT

- An option to play another match.

PROJECT OBJECTIVES:

The major objectives of the Rock · Paper · Scissors project are:

1. Interactive Command-Line Game

To develop an interactive terminal-based game where the player can communicate with the program through keyboard input.

2. Target-Based Match System

To allow the player to select the number of points required to win a match. The program accepts only a positive integer as the target score.

3. Random AI Decision

To generate the computer's choice randomly using Python's random module so that the AI independently selects Rock, Paper, or Scissors.

4. Score Management

To maintain separate scores for the human player and AI and increase the appropriate score whenever a player wins a round.

5. Game Rule Implementation

To correctly implement the standard rules:

- Rock defeats Scissors.

VITYARTHI – PROJECT

- Paper defeats Rock.
- Scissors defeats Paper.
- Same choices result in a tie.

6. Input Validation

To prevent incorrect inputs from terminating the game unexpectedly and provide clear messages when invalid values are entered.

7. User-Controlled Exit

To allow the user to quit before or during a match.

8. Replay Functionality

To allow the player to start another match after completing or abandoning the current match.

FUNCTIONAL REQUIREMENTS:

1. Target Score Input

The system must ask the user to enter the target points required to win.

The input is stripped of unnecessary spaces and checked for the quit command.

If the user enters q, the program exits.

If the input is not a valid integer, an error message is displayed.

VITYARTHI – PROJECT

If the number is zero or negative, the program requests another value.

2. Player Choice

During each round, the player must select one of:

- 1 – Rock
- 2 – Paper
- 3 – Scissors
- Q – Quit

The program validates the entered option before continuing.

FUNCTIONAL REQUIREMENTS – CONTINUED

3. AI Choice Generation

After receiving a valid player choice, the program converts the numeric input into the corresponding choice.

The computer then randomly selects one option from:

rock

paper

scissor

The Python `random.choice()` function is responsible for this selection.

VITYARTHI – PROJECT

4. Round Result Calculation

The program compares the player's choice with the AI's choice.

If both choices are equal:

It's a Tie!

is displayed.

If the player selects a winning combination, the player's score is increased by one.

Otherwise, the AI score is increased by one.

5. Score Display

Before every round, the program displays the current scores:

Score -> You: 2 | AI: 1

The target score is also displayed so that the player can understand the current match status.

6. Match Completion

The match continues while neither player has reached the target:

while point_hi < target_points and point_ai < target_points:

Once one of them reaches the target, the loop terminates and the final winner is displayed.

VITYARTHI – PROJECT

7. Quit Functionality

The player can quit by entering q or quit during a match.

The program displays:

Match abandoned. Goodbye!

and returns from the game function.

8. Replay Functionality

After a match, the program asks:

Play another match? (y/n):

If the user enters y or yes, another match starts. Otherwise, the program displays a thank-you message and terminates.

NON-FUNCTIONAL REQUIREMENTS:

1. Usability

The program should be easy to understand and operate using simple keyboard commands.

2. Reliability

Invalid inputs should be handled without unnecessarily terminating the active game.

3. Performance

The game performs only small in-memory calculations and does not require heavy processing.

VITYARTHI – PROJECT

4. Portability

The project uses standard Python functionality and does not depend on external packages.

5. Maintainability

The use of functions such as `get_target_points()` and `play_rock_paper_scissors()` makes the program easier to understand and modify.

SYSTEM ARCHITECTURE:

VITYARTHI – PROJECT

The project follows a simple layered flow suitable for a console-based game.

1. User Interaction Layer

The user interacts with the program through the command line.

The program uses Python's `input()` function to collect:

- Target score
- Player choice
- Replay decision

2. Input Validation Layer

The program checks whether the entered data is valid.

For target points, the program converts the input to an integer and checks whether it is greater than zero.

For game choices, it checks whether the entered command exists in the `choices_map` dictionary.

3. Game Control Layer

The `play_rock_paper_scissors()` function controls the overall match.

It initializes scores, receives player input, generates AI choices, determines round results, updates scores, and decides when the match ends.

VITYARTHI – PROJECT

4. Random Decision Layer

The computer's move is generated using:

```
random.choice(["rock", "paper", "scissor"])
```

This provides the AI with an independent random selection for every valid round.

5. Result Processing Layer

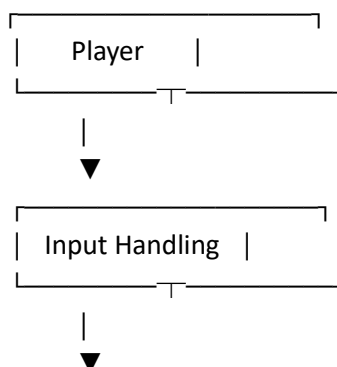
The program compares the two choices and determines whether:

- The round is a tie.
- The player wins.
- The AI wins.

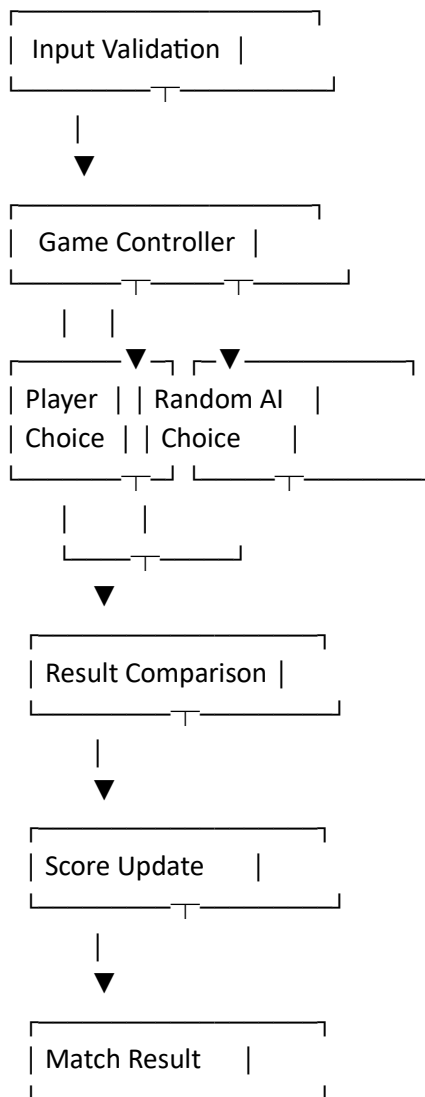
6. Output Layer

The results, scores, error messages, and final match status are displayed directly in the terminal.

ARCHITECTURE FLOW



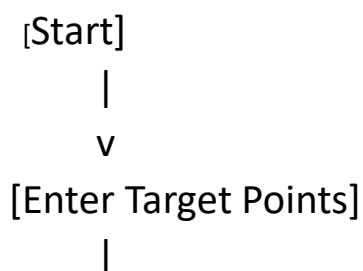
VITYARTHI – PROJECT



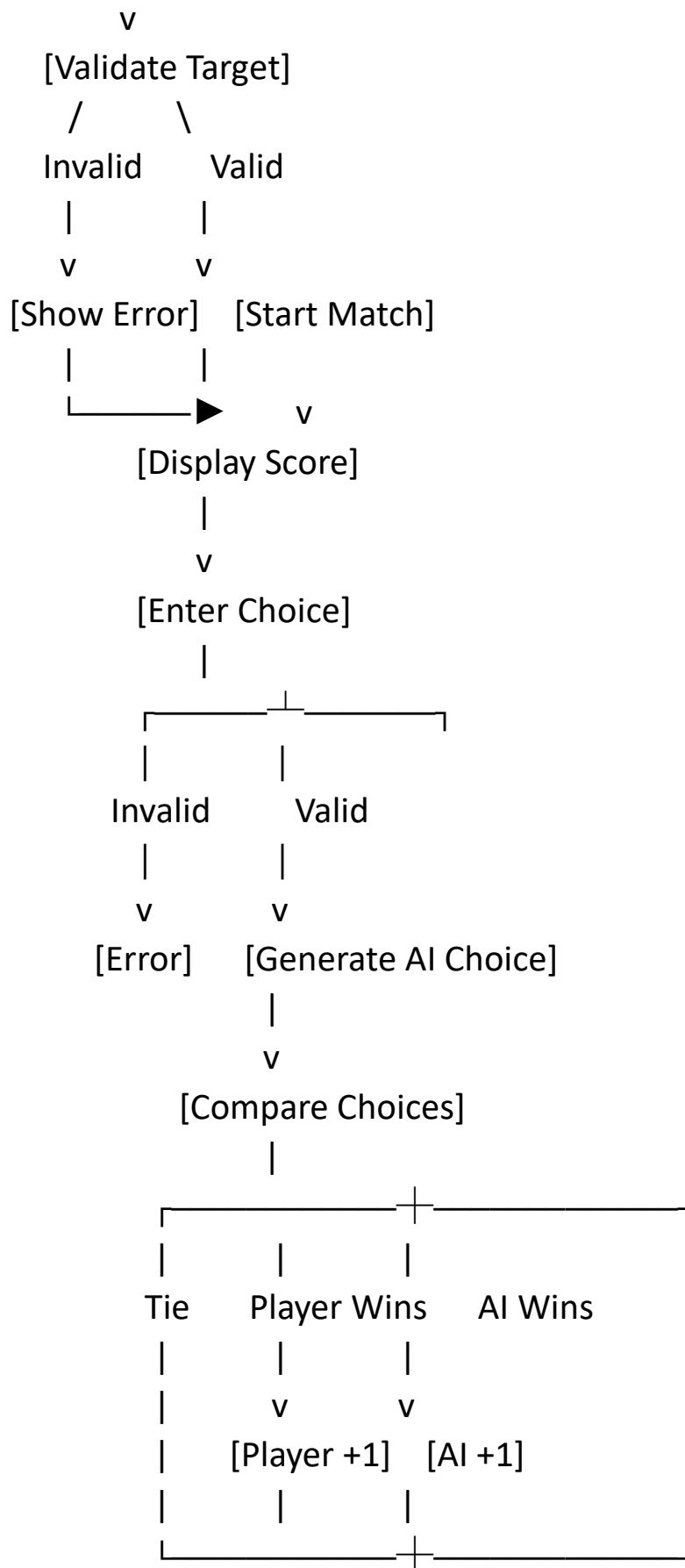
DESIGN DIAGRAMS:

DESIGN DIAGRAMS

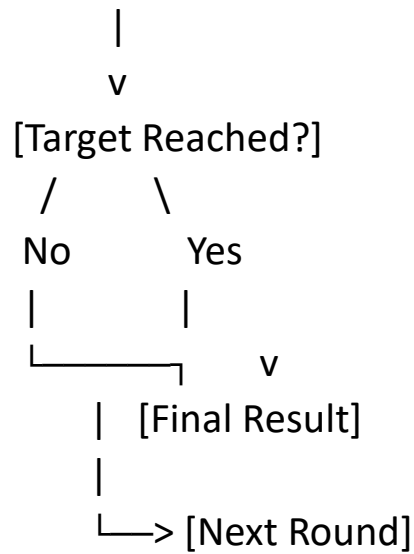
1. WORKFLOW DIAGRAM



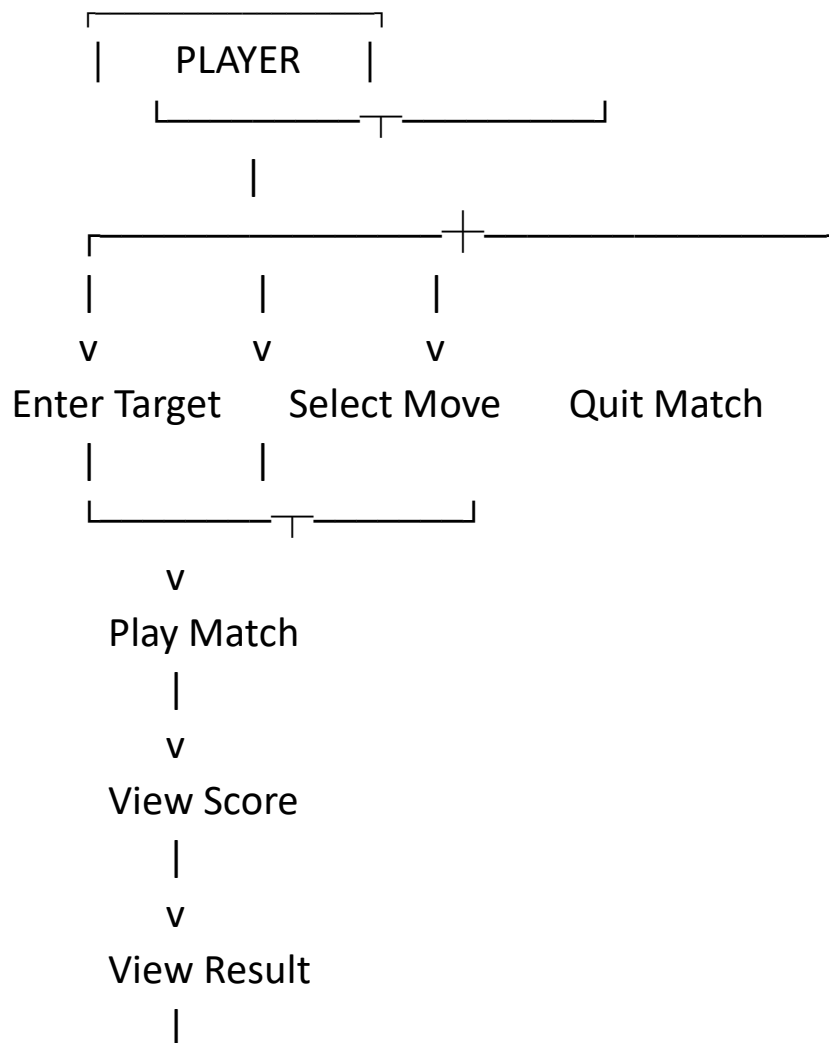
VITYARTHI – PROJECT



VITYARTHI – PROJECT



2. USE CASE DIAGRAM



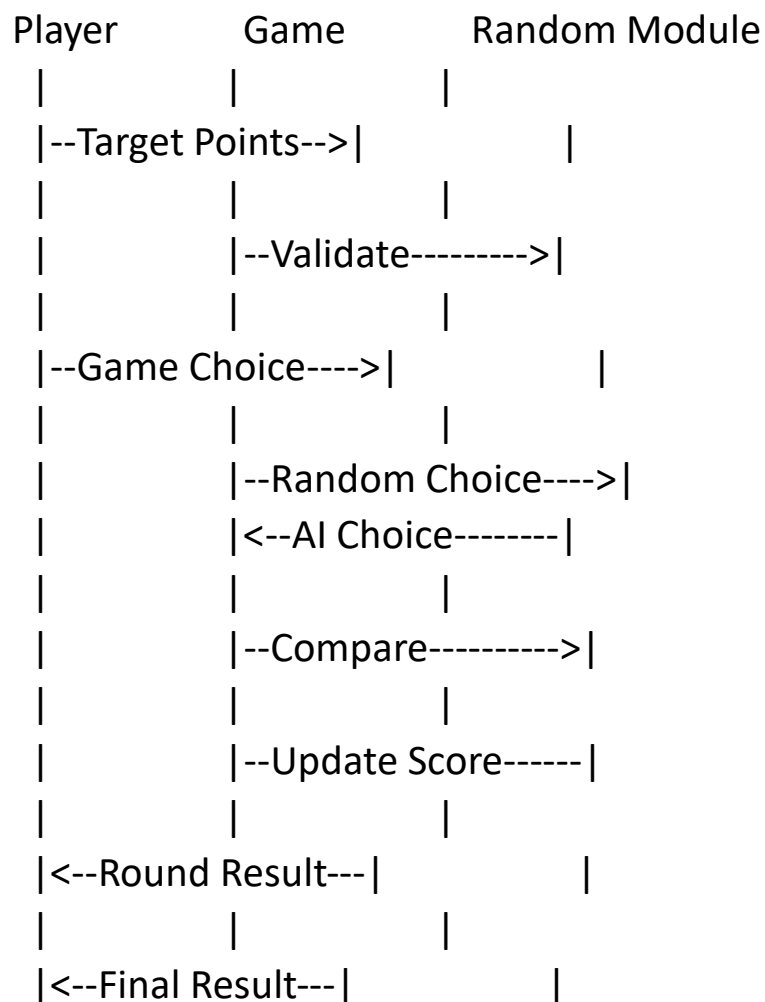
VITYARTHI – PROJECT

v

Play Again

The primary actor is the player. The computer-controlled AI acts as the opponent by generating a random move.

3. SEQUENCE DIAGRAM



VITYARTHI – PROJECT

4. DATA FLOW ARCHITECTURE

Input Stage

The program first collects the target points from the user.

During the game, it repeatedly collects the player's choice.

Validation Stage

The target input is converted into an integer using `int()` inside a try-except block.

The game choice is checked against the `choices_map` dictionary.

Choice Mapping Stage

The numeric choices are mapped as:

```
choices_map = {  
    "1": "rock",  
    "2": "paper",  
    "3": "scissor"  
}
```

This allows the program to use meaningful text values internally.

AI Generation Stage

The AI selects randomly from the three available choices.

VITYARTHI – PROJECT

Comparison Stage

The player's selection and AI selection are compared using conditional expressions.

Score Stage

The score of the round winner is increased by one.

Presentation Stage

The selected choices, current score, round result, and final result are printed to the terminal.

COMPONENT BREAKDOWN

`get_target_points()`

Responsible for obtaining and validating the target score.

`play_rock_paper_scissors()`

Main function responsible for running a complete match.

`random`

Python standard library module used for random AI selection.

`sys`

Python standard library module used for exiting the program when required.

VITYARTHI – PROJECT

choices_map

Dictionary used to convert numerical commands into game choices.

icons

Dictionary used to associate game choices with visual symbols.

DECISIONS & RATIONALE:

1. Programming Language: Python

Python was selected for implementing the project because the game requires straightforward input handling, conditional logic, loops, functions, and random selection.

The syntax is concise and allows the core game logic to be implemented in a relatively small source file.

2. Interface: Command-Line Interface

A command-line interface was selected because the project focuses on demonstrating programming fundamentals rather than graphical interface development.

The terminal provides all required functionality through simple keyboard input.

VITYARTHI – PROJECT

3. Function-Based Structure

The program separates the target-point input logic from the main game logic.

The `get_target_points()` function handles target validation, while `play_rock_paper_scissors()` handles the actual match.

This separation improves readability and makes individual parts easier to modify.

4. Dictionary-Based Choice Mapping

The program uses a dictionary to map the user's numerical commands to the corresponding game choices.

```
choices_map = {  
    "1": "rock",  
    "2": "paper",  
    "3": "scissor"  
}
```

This avoids repeatedly writing separate conversion conditions.

5. Random AI Selection

The AI selection uses the Python random module:

```
ai_choice = random.choice(  
    ["rock", "paper", "scissor"]  
)
```

VITYARTHI – PROJECT

This provides a simple mechanism for generating the computer's move.

6. Target-Based Loop

Instead of playing a fixed number of rounds, the program continues until either player reaches the selected target.

This makes the length of each match dependent on the selected target and round results.

7. In-Memory Score Management

The game stores the scores in variables:

```
point_hi = 0
```

```
point_ai = 0
```

No database or external storage is required because the score is relevant only to the current match.

TESTING APPROACH:

The project can be tested through several categories.

1. Valid Input Testing

Test valid target values such as:

1

3

VITYARTHI – PROJECT

5
10

The program should accept positive integers.

2. Invalid Target Testing

Test:

0
-1
abc
3.5

The program should reject invalid target values.

The source code specifically catches ValueError when the target cannot be converted to an integer.

3. Choice Validation Testing

Test:

1
2
3

These should produce valid game choices.

Test invalid values:

4
5
abc

VITYARTHI – PROJECT

The program should display an invalid-selection message.

4. Tie Testing

Test situations where both player and AI select the same choice.

Expected result:

It's a Tie!

5. Player Win Testing

Test the three player-winning combinations:

Rock vs Scissor

Paper vs Rock

Scissor vs Paper

The player's score should increase.

6. AI Win Testing

When the AI has a winning combination, the AI score should increase.

7. Quit Testing

Test:

q

quit

during a match.

VITYARTHI – PROJECT

The match should be abandoned.

8. Replay Testing

After a completed match, test:

y

yes

n

to verify the replay behaviour.

TEST CASES:

Test Case	Input / Condition	Expected Result
TC01	Target = 5	Match starts with target 5
TC02	Target = 0	Error message displayed
TC03	Target = -2	Error message displayed
TC04	Target = abc	Invalid input message
TC05	Choice = 1	Rock selected
TC06	Choice = 2	Paper selected
TC07	Choice = 3	Scissors selected
TC08	Choice = 4	Invalid selection
TC09	Choice = abc	Invalid selection
TC10	Player = Rock, AI = Scissor	Player receives point
TC11	Player = Paper, AI = Rock	Player receives point
TC12	Player = Scissor, AI = Paper	Player receives point
TC13	Same player and AI choice	Tie; no point added
TC14	Player reaches target	Player wins match
TC15	AI reaches target	AI wins match
TC16	User enters q	Current match is abandoned
TC17	User enters quit	Current match is abandoned
TC18	Replay = y	New match starts
TC19	Replay = yes	New match starts
TC20	Replay = n	Program terminates

VITYARTHI – PROJECT

CHALLENGES FACED:

1. Handling User Input

One of the important challenges in a command-line game is ensuring that unexpected input does not cause the program to terminate.

The target-point function therefore uses try-except to handle invalid integer input.

2. Validating Game Choices

The program must distinguish between valid choices and invalid commands.

The dictionary-based validation ensures that only 1, 2, and 3 are accepted as game choices.

3. Random AI Behaviour

The AI needs to make a choice independently during every valid round.

The project solves this using Python's `random.choice()` function.

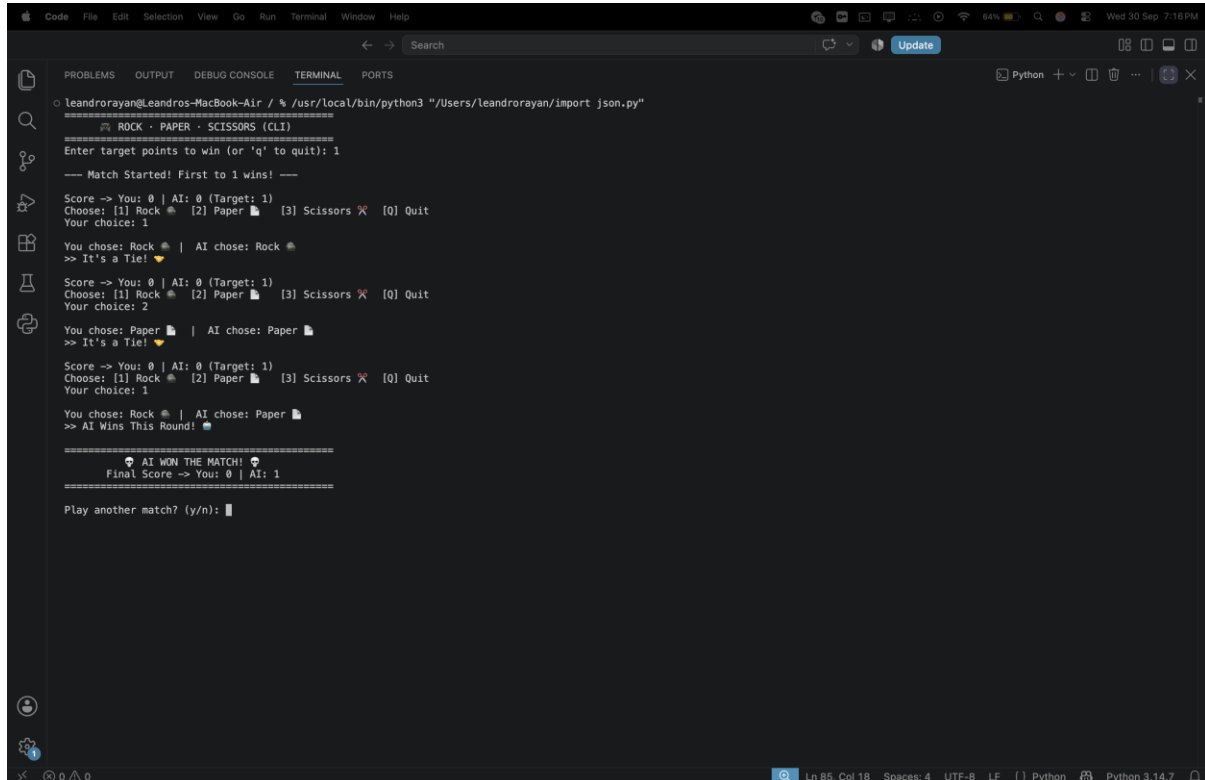
4. Correct Implementation of Game Rules

The Rock–Paper–Scissors rules require several conditional combinations.

The program explicitly checks the three combinations in which the player wins. If the choices are not equal and none of those conditions are true, the AI

VITYARTHI – PROJECT

OUTPUT:



```
leandrorayan@leandros-MacBook-Air / % /usr/local/bin/python3 "/Users/leandrorayan/import json.py"
=====
ROCK - PAPER - SCISSORS (CLI)
=====
Enter target points to win (or 'q' to quit): 1
--- Match Started! First to 1 wins! ---
Score -> You: 0 | AI: 0 (Target: 1)
Choose: [1] Rock [2] Paper [3] Scissors [Q] Quit
Your choice: 1
You chose: Rock | AI chose: Rock
>> It's a Tie!
Score -> You: 0 | AI: 0 (Target: 1)
Choose: [1] Rock [2] Paper [3] Scissors [Q] Quit
Your choice: 2
You chose: Paper | AI chose: Paper
>> It's a Tie!
Score -> You: 0 | AI: 0 (Target: 1)
Choose: [1] Rock [2] Paper [3] Scissors [Q] Quit
Your choice: 1
You chose: Rock | AI chose: Paper
>> AI Wins This Round!
=====
AI WON THE MATCH!
Final Score -> You: 0 | AI: 1
=====
Play another match? (y/n):
```

REFERENCES:

- Python Software Foundation. **Python 3 Documentation**. Python.org.
- Python Software Foundation. **random — Generate pseudo-random numbers**. Python Standard Library Documentation.
- Python Software Foundation. **sys — System-specific parameters and functions**. Python Standard Library Documentation.
- Python Software Foundation. **Built-in Functions — input() and print()**. Python Documentation.

VITYARTHI – PROJECT

- Python Software Foundation. **Errors and Exceptions — try and except.** Python Documentation.
- Python Software Foundation. **Data Structures — Dictionaries and Lists.** Python Documentation.
- Python Software Foundation. **Defining Functions in Python.** Python Documentation.
- **Rock Paper Scissors — Game Rules and Gameplay Concepts**, general reference material used for understanding the rules of the game.